DS unit-1:

(1a)Structure definition: A structure in C is a user-defined data type that allows you to combine different types of data items under one name.
It is useful when you want to store related information together.

SYNTAX

```
struct structure_name {
    data_type member1;
    data_type member2;
    ...
};
```

Example:

```
struct Student {
    int roll;
    char name[50];
    float marks;
};
```

Creating Structure Variable:
After structure definition, we have to create variable of that structure to use it.

We can create structure variable in two ways–
 create a structure variable separately.
 create a structure variable along with the definition.

```
struct strcuture_name var;
struct A {
int x;
};
struct A a;
```

We can also declare structure variables with structure definition–

```
struct structure_name {
…
}var1, var2….;
```

Example :

```
struct A{
int x;
}a;
```

Access Structure Members:
To access or modify members of a structure, we use the (.) dot operator.
This is applicable

when we are using structure variables directly.

SYNTAX:

```
structure_variable_name . member1;
strcuture_variable_name . member2;
```

Example :

```
struct student{
char name[10];
int rollno;
int marks;
};
struct student s;
s.name;
s.rollno;
s.marks;
```

Initialization of Structure :

we can initialize in two methods

Method 1: At Declaration

```
struct Student s1 = {101, "Harshitha", 92.5};
```

Values are assigned in the same order as members.

Method 2: Using Dot Operator

```
struct Student s1;
s1.roll = 101;
strcpy(s1.name, "Harshitha");
s1.marks = 92.5;
```

(1b) program:

```
#include <stdio.h>

struct Student {
    int roll;
    char name[50];
    float marks;
};

int main() {
    struct Student s;

    // Reading student details
    printf("Enter Roll Number: ");
    scanf("%d", &s.roll);

    printf("Enter Name: ");
    scanf("%s", s.name);

    printf("Enter Marks: ");
    scanf("%f", &s.marks);
```

```
    // Displaying student details
    printf("\n--- Student Details ---\n");
    printf("Roll Number: %d\n", s.roll);
    printf("Name: %s\n", s.name);
    printf("Marks: %.2f\n", s.marks);

    return 0;
}
```

```
output:Enter Roll Number: 101
Enter Name: Harshitha
Enter Marks: 92.5

--- Student Details ---
Roll Number: 101
Name: Harshitha
Marks: 92.50
```

(2a)

Nested Structure in C:

A nested structure in C is a structure within structure. One structure can be declared inside another structure in the same way structure members are declared inside a structure.

The structure can be nested in the following different ways:

1. By separate nested structure
2. By embedded nested structure

Example:

```
Struct address
{
int doorno;
char city[50];
int pincode;
};
Struct student
{
int rollno;
char name[20];
struct address a1;
};
# include <stdio.h>
#include <string.h>
int main( )
struct student s1;
s1.rollno=45;
strcpy(s1.name,”harshitha”);
s1.a1.doorno=8/16;
strcpy(s1.a1.city,”Vijayawada”);
s1.a1.pincode = 522330;
```

```
printf(“ %d %s %d %s %d
“,s1.rollno,s1.name,s1.a1.doorno,s1.a1.city,s1.a1.pincode);
return 0;
}
```
Output :

45 harshitha 8/16 Vijayawada 522330

(2b) typedef keyword:
The typedef is a keyword that is used to provide existing data types with a new name. In C typedef keyword is used to redefine the name of already existing data types. When names of datatypes become difficult to use in programs, typedef is used with user-defined datatypes.

Syntax of typedef :

```
typedef existing_type new_type;
```

where,
 existing_type: The type that we want to change (e.g., int, float, struct, etc.).
 new_type: The new name for the existing type.

Example program:

```
  #include <stdio.h>
struct student
{
   int roll;
char name [20];
};
int main ()
{
typedef struct student s
s s1;              // no need to write struct
s1.roll = 45;
printf("%d", s1.roll);
return 0;
}
```
Output :
45

(3a) ARRAY OF STRUCTURES :
An array of structures is simply an array where each element is a structure. It allows you to store several structures of the same type in a single array.
An array of structures is needed when we want to store details of multiple records of the same type.
we need array of structures because :

*To store data of many objects
*To organize related information efficiently
*To access records using index numbers
*To avoid creating separate variables for each record

Example program:

```
#include <stdio.h>

struct Student {
    int roll;
    char name[50];
    float marks;
};

int main() {
    struct Student s[3];
    int i;

    // Reading details
    for(i = 0; i < 3; i++)
 {
        printf("\nEnter details of student %d\n", i + 1);
        printf("Roll Number: ");
        scanf("%d", &s[i].roll);

        printf("Name: ");
        scanf("%s", s[i].name);

        printf("Marks: ");
        scanf("%f", &s[i].marks);
    }

    // Displaying details
    printf("\n--- Student Details ---\n");
    for(i = 0; i < 3; i++)
 {
        printf("\nStudent %d\n", i + 1);
        printf("Roll Number: %d\n", s[i].roll);
        printf("Name: %s\n", s[i].name);
        printf("Marks: %.2f\n", s[i].marks);
    }

    return 0;
}
```

output:

```
Enter details of student 1
Roll Number: 101
Name: Harshitha
Marks: 92.5
```

```
Enter details of student 2
Roll Number: 102
Name: Ramesh
Marks: 85.0

Enter details of student 3
Roll Number: 103
Name: Anjali
Marks: 88.75

--- Student Details ---

Student 1
Roll Number: 101
Name: Harshitha
Marks: 92.50

Student 2
Roll Number: 102
Name: Ramesh
Marks: 85.00

Student 3
Roll Number: 103
Name: Anjali
Marks: 88.75
```

(3b)

```
#include <stdio.h>

struct Student {
    int roll;
    char name[50];
    float m1, m2, m3;
};

float Total(struct Student s) {
    return s.m1 + s.m2 + s.m3;
}

float Average(struct Student s) {
    return (s.m1 + s.m2 + s.m3) / 3;
}

int main() {
    struct Student s;
    float total, average;

    printf("Enter Roll Number: ");
    scanf("%d", &s.roll);
```

```
    printf("Enter Name: ");
    scanf("%s", s.name);

    printf("Enter Marks of 3 subjects:\n");
    scanf("%f %f %f", &s.m1, &s.m2, &s.m3);

    total = Total(s);
    average = Average(s);

    printf("Roll Number: %d\n", s.roll);
    printf("Name: %s\n", s.name);
    printf("Total Marks: %.2f\n", total);
    printf("Average Marks: %.2f\n", average);

    return 0;
}
```

output:

```
Enter Roll Number: 101
Enter Name: Harshitha
Enter Marks of 3 subjects:
85
90
95

Roll Number: 101
Name: Harshitha
Total Marks: 270.00
Average Marks: 90.00
```

(4a) DIFFERENCE BETWEEN ARRAY, STRUCTURE AND UNION IN C

note:[idi ila kakunda table format lo rayali like manam assignment lo rasam kadha]

1. Definition

```
Array      : Collection of elements of same data type stored in contiguous
memory.
Structure  : User-defined data type that groups different data types into one
unit.
Union      : User-defined data type that stores different data types in the
same memory location.
```

2. Keyword

```
Array      : No special keyword (uses data type like int, float).
Structure  : Defined using 'struct' keyword.
Union      : Defined using 'union' keyword.
```

3. Data Types

Array   : Stores only same type of elements.
Structure : Stores different types of members.
Union   : Stores different types of members.

4. Size
Array   : Size = number of elements × size of data type.
Structure : Size = sum of sizes of all members
Union   : Size = size of largest member

5. Memory Allocation
Array   : Each element has separate memory location.
Structure : Each member has separate memory location.
Union   : All members share same memory location.

6. Data Overlap
Array   : No overlap.
Structure : No overlap.
Union   : Full overlap .

7. Accessing
Array   : Accessed using index (arr[0]).
Structure : Accessed using dot operator (s.roll).
Union   : Accessed using dot operator (u.x), but only one member holds value at a time.

Array example :

```
 #include <stdio.h>
int main() {
    int a[3] = {10, 20, 30};
    printf("%d %d %d", a[0], a[1], a[2]);
    return 0;
}
```

Output : 10 20 30

Sruct example:

```
 #include <stdio.h>

struct Student {
    int roll;
    char name[20];
    float marks;
};

int main() {
    struct Student s1 = {1, "Ravi", 85.5};
    printf("%d %s %.1f", s1.roll, s1.name, s1.marks);
    return 0;
}
```

Output: 1 Ravi 85.5

Union example:

```
#include <stdio.h>

union Data {
    int i;
    float f;
};

int main() {
    union Data d;
    d.i = 10;
    printf("%d\n", d.i);

    d.f = 3.5;
    printf("%.1f", d.f);
    return 0;
}
```

Output : 10 3.5

(4b)

```
#include <stdio.h>
Enum week{mon,tue,wed,thu,fri,sat,sun};
int main ()
{
Enum week day;
day = wed;
printf("%d",day);
return 0;
}
```

output :

2

(5a)

Self referential definition :

Self referential structure is a structure that contains pointer to the same type of the structre inside it.

Syntax:

```
Struct structure_name
{
Datatype member1;
Datatype member2;
Struct structure_name pointer_name;
}
```

Example program :

```
Struct student
{
int rollno;
char gen;
struct student *p;
}
```

```
#include <stdio.h>
int main()
{
struct student s1;
s1.rollno = 45;
s1.gen = ‘f’;
s1.p = &s1;
printf(“%d”,s1.rollno);
printf(“%d”,s1.p->rollno);
return 0;
}
Output : 45 45

(5b)
#include <stdio.h>

struct Student {
    int roll;
    char name[50];
    float marks;
};

int main() {
    struct Student s[5];
    int i;

    for(i = 0; i < 5; i++) {
        printf("\nEnter details of student %d\n", i + 1);

        printf("Roll Number: ");
        scanf("%d", &s[i].roll);

        printf("Name: ");
        scanf("%s", s[i].name);

        printf("Marks: ");
        scanf("%f", &s[i].marks);
    }

     for(i = 0; i < 5; i++)
 {
        printf("\nStudent %d\n", i + 1);
        printf("Roll Number: %d\n", s[i].roll);
        printf("Name: %s\n", s[i].name);
        printf("Marks: %.2f\n", s[i].marks);
    }

    return 0;
}
output :
```

```
Enter details of student 1
Roll Number: 101
Name: Harshitha
Marks: 92.5

Enter details of student 2
Roll Number: 102
Name: Ramesh
Marks: 85

Enter details of student 3
Roll Number: 103
Name: Anjali
Marks: 88

Enter details of student 4
Roll Number: 104
Name: Ravi
Marks: 90

Enter details of student 5
Roll Number: 105
Name: Sneha
Marks: 95

Student 1
Roll Number: 101
Name: Harshitha
Marks: 92.50

Student 2
Roll Number: 102
Name: Ramesh
Marks: 85.00

Student 3
Roll Number: 103
Name: Anjali
Marks: 88.00

Student 4
Roll Number: 104
Name: Ravi
Marks: 90.00

Student 5
Roll Number: 105
Name: Sneha
Marks: 95.00
```

(6a)
Write a short notes on following terms
i) Enum ii) typedef iii) Bit fields

i) Enum (Enumeration)
An enum is a user-defined data type in C used to assign names to a set of integer constants. It makes the program more readable and easy to maintain.
Syntax:

```
enum enum_name {
    constant1, constant2, constant3
};
```

Example:

```
#include <stdio.h>
enum week {Sun, Mon, Tue, Wed, Thu, Fri, Sat};

int main() {
    enum week day;
    day = Wed;
    printf("%d", day);
}
```

output :
2

ii) Typedef
The typedef is a keyword that is used to provide existing data types with a new name. In C typedef keyword is used to redefine the name of already existing data types. When names of datatypes become difficult to use in programs, typedef is used with user-defined datatypes.

Syntax of typedef :

```
typedef existing_type new_type;
```

where,
 existing_type: The type that we want to change (e.g., int, float, struct, etc.).
 new_type: The new name for the existing type.

Example program:

```
  #include <stdio.h>
struct student
{
   int roll;
char name [20];
};
int main ()
{
typedef struct student s
```

```
s s1;              // no need to write struct
s1.roll = 45;
printf("%d", s1.roll);
return 0;
}
Output :
45
```

iii) Bit Fields

Bit fields allow storing data in a specific number of bits within a structure. This helps in saving memory when variables require only a few bits.

```
Syntax:
struct struct_name {
    data_type variable : number_of_bits;
};

Example:
#include <stdio.h>

struct data {
    unsigned int x : 2;
    unsigned int y : 3;
};

int main() {
    struct data d;
    d.x = 2;
    d.y = 5;
    printf("%d %d", d.x, d.y);
}
Output :
2 5

(6b)
#include <stdio.h>

struct Student
{
    int roll;
    char name[50];
    float marks;
};

int main()
{
    struct Student s;

    printf("Enter roll number: ");
```

```
    scanf("%d", &s.roll);

    printf("Enter name: ");
    scanf("%s", s.name);

    printf("Enter marks: ");
    scanf("%f", &s.marks);

    printf("\nStudent Details:\n");
    printf("Roll Number: %d\n", s.roll);
    printf("Name: %s\n", s.name);
    printf("Marks: %.2f\n", s.marks);

    return 0;
}
```

output:
Enter roll number: 101
Enter name: Harshitha
Enter marks: 89.5

Student Details:
Roll Number: 101
Name: Harshitha
Marks: 89.50

(7a)
Explain the classification of data structures with suitable examples
Classification of Data Structures
A Data Structure is a way of organizing and storing data so that it can be accessed and modified efficiently.
Data structures are mainly classified into two types:

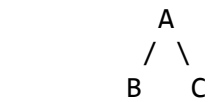
1) Primitive Data Structures
Primitive data structures are basic data types provided by the programming language to store single values.
Examples:
- int – stores integers
- float – stores decimal numbers
- char – stores characters
- double – stores large decimal numbers

Example in C:
```
int a = 10;
float b = 5.5;
char ch = 'A';
```
These are simple and built-in data types.

2) Non-Primitive Data Structures
Non-primitive data structures are derived from primitive data types and can store multiple values.

They are further divided into two types:
a) Linear Data Structures
In linear data structures, elements are arranged sequentially (one after another).
Examples:
• Array
• Linked List
• Stack
• Queue
Example:
Array: 10 20 30 40

b) Non-Linear Data Structures
In non-linear data structures, elements are not arranged sequentially. They form hierarchical or network relationships.
Examples:
• Tree
• Graph
Example:
Tree:



(7b)
iii) Bit Fields
Bit fields allow storing data in a specific number of bits within a structure. This helps in saving memory when variables require only a few bits.

Syntax:
```
struct struct_name {
    data_type variable : number_of_bits;
};
```

Example:
```
#include <stdio.h>

struct data {
    unsigned int x : 2;
    unsigned int y : 3;
};

int main() {
    struct data d;
    d.x = 2;
    d.y = 5;
    printf("%d %d", d.x, d.y);
}
```

Output :
2 5

(8a)
Demonstrate the concept of structures and functions
Structures and Functions in C
A structure in C is a user-defined data type used to store different types of data elements under one name. When a structure is used with functions, the structure variables can be passed to the functions to perform operations like displaying or modifying the data.

Ways to Use Structures with Functions
1. Passing structure to a function
2. Returning structure from a function

Example Program

```
#include <stdio.h>

struct Student
{
    int roll;
    char name[20];
    float marks;
};

void display(struct Student s)
{
    printf("Roll Number: %d\n", s.roll);
    printf("Name: %s\n", s.name);
    printf("Marks: %.2f\n", s.marks);
}

int main()
{
    struct Student s1 = {101, "Ravi", 85.5};
    display(s1);   // passing structure to function
    return 0;
}
```

Output:
Roll Number: 101
Name: Ravi
Marks: 85.50

(8b)

```
#include <stdio.h>
#include <string.h>

struct Book {
```

```
    char title[50];
    char author[50];
    float price;
};

int main() {
    struct Book b;

    printf("Enter Book Title: ");
    scanf("%s", b.title);

    printf("Enter Author Name: ");
    scanf("%s", b.author);

    printf("Enter Price: ");
    scanf("%f", &b.price);

    printf("\n--- Book Details ---\n");
    printf("Title  : %s\n", b.title);
    printf("Author : %s\n", b.author);
    printf("Price  : %.2f\n", b.price);

    return 0;
}
output
Enter Book Title: Cprogramming
Enter Author Name: Ritchie
Enter Price: 450

--- Book Details ---
Title  : Cprogramming
Author : Ritchie
Price  : 450.00

(9a)
Define union. Explain with suitable examples
Union in C
A union is a user-defined data type in C that allows storing different types
of data in the same memory location.
Unlike structures, all members of a union share the same memory.

Syntax
union union_name
{
    data_type member1;
    data_type member2;
};
```
Union example:
```
#include <stdio.h>
```

```
union Data {
    int i;
    float f;
};

int main() {
    union Data d;
    d.i = 10;
    printf("%d\n", d.i);

    d.f = 3.5;
    printf("%.1f", d.f);
    return 0;
}
Output : 10 3.5
```

Explanation
- union Data contains int, float, and char array members.
- All members share the same memory location.
- When one member is assigned a value, the previous value may be overwritten.

Key Points
- Memory is shared by all members.
- Only one member can store a value at a time.
- Size of union = size of the largest member.
- Used when memory optimization is required.

(9b)

```
#include <stdio.h>

struct Address {
    char city[20];
    int pincode;
};

struct Employee {
    int id;
    char name[20];
    struct Address a;   // Address inside Employee
};

int main() {
    struct Employee e;

    printf("Enter ID: ");
    scanf("%d", &e.id);

    printf("Enter Name: ");
```

```
    scanf("%s", e.name);

    printf("Enter City: ");
    scanf("%s", e.a.city);

    printf("Enter Pincode: ");
    scanf("%d", &e.a.pincode);

    printf("\nEmployee Details\n");
    printf("ID: %d\n", e.id);
    printf("Name: %s\n", e.name);
    printf("City: %s\n", e.a.city);
    printf("Pincode: %d\n", e.a.pincode);

    return 0;
}

output:
Enter ID: 1
Enter Name: Ravi
Enter City: Hyderabad
Enter Pincode: 500001

Employee Details
ID: 1
Name: Ravi
City: Hyderabad
Pincode: 500001
```